

TP1 - Reparer une image Docker cassee

Niveau 1 - Analyse - 30 min - Facile

- Quelle ligne est en cause ?
 - Ligne 1 : `FROM node:18`
- Quel est le problème exact ?
 - L'image `node:18` est une image basée sur Debian et elle fait environ 900 MB
- Quelle est la conséquence en production ?
 - L'image finale de node dépasse de très loin les 200 MB demander, les déploiements sont lents, et la consommation de stockage devient élevée de façon inutile.

Les 5 problèmes distincts à trouver.

Ligne(s)	Problème observé	Conséquence en production	Correction envisagée
1	<code>FROM node:18</code> image Debian complète	Image supérieur à 900 MB, déploiements lents, stockage qui devient consommé de manière inutile	Remplacer par <code>node:18-alpine</code>
6-8	<code>ENV</code> <code>API_KEY</code> , <code>DB_PASSWORD</code> , <code>DB_HOST</code> écrits en clair / dur	Secrets afficher de manière non sécuriser en dur dans le code et visible via <code>docker inspect</code> ou <code>docker history</code> , fuite de données si l'image est partagée	Supprimer ces lignes, passer les secrets au runtime via <code>--env-file</code>
11 + 15	<code>COPY . .</code> effectué avant <code>RUN npm install</code>	Cache Docker invalidé à chaque modification du code source, npm réinstallé entièrement à chaque build	Copier <code>package*.json</code> en premier, installer les dépendances, puis copier le code
19-25	<code>apt-get install vim git htop wget curl net-tools</code>	Image surchargée, surface d'attaque agrandie, bloc qui ferait planter le build sur alpine	Supprimer entièrement ce bloc

Ligne(s)	Problème observé	Conséquence en production	Correction envisagée
		car <code>apt-get</code> n'existe pas	
NULL	Aucune directive <code>USER</code> container tourne en root	En cas de compromission, l'attaquant dispose des droits root dans le container	Ajouter <code>USER node</code> avant le CMD

Niveau 2 - Reparation - 45 min - Intermediaire

Objectifs a atteindre

- Image de base passée à `node:18-alpine`
- Suppression des 3 `ENV` contenant les secrets
- `COPY package*.json` → `RUN npm install --omit=dev` → `COPY index.js` (cache optimisé, devDependencies exclues)
- Suppression du `RUN npm run build` et du bloc `apt-get`
- Ajout `RUN mkdir -p /app/data && chown -R node:node /app` pour que l'utilisateur node puisse écrire
- Ajout `USER node`
- CMD corrigé vers `["node", "index.js"]`

`.dockerignore` complet créé contenant : `node_modules/` , `.env` , `*.log` , `.git/` , `dist/` , `tp1-enonce.docx`

TP1 — Image Docker

Messages (1)

test (22/06/2026 07:48:57)

Container : c17a9edc2fb1

```
r0r0000000@Skz:/mnt/c/Users/R0r0000000/Desktop/TP DOCKER/SujetTP1$ sudo
docker build -t tp1:corrige .
[+] Building 0.4s (11/11) FINISHED
docker:default
=> [internal] load build definition from Dockerfile
0.0s
=> => transferring dockerfile: 242B
```

```

0.0s
=> [internal] load metadata for docker.io/library/node:18-alpine
0.2s
=> [internal] load .dockerignore
0.0s
=> => transferring context: 93B
0.0s
=> [1/6] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28f
d3ebc498332f249011d118945588d0a35cb9bc4b8c 0.0s
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28f
d3ebc498332f249011d118945588d0a35cb9bc4b8c 0.0s
=> [internal] load build context
0.0s
=> => transferring context: 62B
0.0s
=> CACHED [2/6] WORKDIR /app
0.0s
=> CACHED [3/6] COPY package*.json ./
0.0s
=> CACHED [4/6] RUN npm install --omit=dev
0.0s
=> CACHED [5/6] COPY index.js ./
0.0s
=> CACHED [6/6] RUN mkdir -p /app/data && chown -R node:node /app
0.0s
=> exporting to image
0.1s
=> => exporting layers
0.0s
=> => exporting manifest sha256:b9e6679c6bc5e407c80b4232663112eff752a6
8d63fedb1bf5d45864daea8367 0.0s
=> => exporting config sha256:490a6697bdf6545781e545c07bae05f2760a7e5e
1e72678802d51f0387baa9a6 0.0s
=> => exporting attestation manifest sha256:bf23f497e6d270d3cd141e7144
f18fe5d876b87cd0be3e277501988f40260ce5 0.0s
=> => exporting manifest list sha256:83fba4c62bc0260fabd888fa83f786fa6
9dc6b83fbfe5861395587e4bd50eb84 0.0s
=> => naming to docker.io/library/tp1:corrige
0.0s
=> => unpacking to docker.io/library/tp1:corrige
0.0s
roroooooooo@Skz:/mnt/c/Users/Roroooooooo/Desktop/TP DOCKER/SujetTP1$ sudo
docker images tp1:corrige

```

```

i Info →    U    In Use
IMAGE          ID          DISK USAGE    CONTENT SIZE    EXTRA

```

```

tp1:corrige 83fba4c62bc0 186MB 45.7MB U
rorooooooo@Skz:/mnt/c/Users/Rorooooooo/Desktop/TP DOCKER/SujetTP1$ sudo
docker run --rm -p 3000:3000 tp1:corrige
docker: Error response from daemon: failed to set up container networki
ng: driver failed programming external connectivity on endpoint angry_b
habha (b6f93fdfbe4b2b0a3db213390de5a7de7a9f8e2d6bb9716ac4d0e3deeea60a4
0): Bind for 0.0.0.0:3000 failed: port is already allocated

Run 'docker run --help' for more information
rorooooooo@Skz:/mnt/c/Users/Rorooooooo/Desktop/TP DOCKER/SujetTP1$ sudo
docker run --rm tp1:corrige whoami
node
rorooooooo@Skz:/mnt/c/Users/Rorooooooo/Desktop/TP DOCKER/SujetTP1$ sudo
docker inspect tp1:corrige --format '{{range .Config.Env}}{{println .}}
{{end}}' | grep -iE "API_KEY|DB_PASSWORD|DB_HOST"
rorooooooo@Skz:/mnt/c/Users/Rorooooooo/Desktop/TP DOCKER/SujetTP1$

```